

Project Proposal: Full-Stack To-Do List Web Application

Python Programming — Course Project

Introduction

For my Python Programming project I will build and deploy a full-stack to-do list website using the Django web framework. The application will let a user register an account, log in, and create, view, update, and delete their own tasks, with each user only able to see their own list. I am basing the project on a nine-part video series (“Django Fullstack Todolist Website” by THE PROTON GUY) that walks through the build from setup to deployment. A to-do list is a great first project because the core idea is simple and familiar, but completing it still requires me to practice the fundamentals of Python and apply them to a real, working program rather than isolated exercises. By the end I will have a live website I can share, along with hands-on experience in how a real web application is structured.

Plan of Action

I will follow the playlist in order, treating each video as a milestone and testing my code after every step before moving on:

1. Set up the project: install Python and Django, create a virtual environment, start the Django project and app, and run the development server to confirm it works.
2. Build the templates and views: create the base HTML pages and connect them to Django views and URLs so the site renders in the browser.
3. Add user authentication: build register, login, and logout functionality so each person has their own account.
4. Create tasks: design the Task model and a form so a logged-in user can add new to-do items to the database.
5. Display tasks: query the database and show each user's task list on the page.
6. Update and delete tasks: add the ability to mark a task complete and remove tasks from the list.
7. Restrict access: make sure users can only see and edit their own tasks, not anyone else's.
8. Deploy the website: push the finished project online so it is accessible as a live link, then do final testing and write up my documentation.

Technologies Used

- **Python:** The core programming language for all of the application logic.
- **Django:** A Python web framework that handles URLs, views, forms, the database, and user authentication.
- **SQLite:** The lightweight database (built into Django) that stores user accounts and tasks.
- **HTML & CSS:** Used to build and style the web pages the user sees and interacts with.
- **Git & GitHub:** For version control and backing up my code as I follow each step.
- **Hosting platform:** A deployment service to publish the finished site online as a live, shareable link.

Timeline

As a summer project, I plan to complete the build in 10 days, working on it intensively each day:

Day	Focus	Deliverable
Day 1	Install Python and Django, set up the project and virtual environment	Running Django development server
Day 2	Build templates, views, and URLs	Base pages render in the browser
Day 3	Add user authentication — register and login	Users can create an account and log in
Day 4	Finish authentication — logout and session handling	Complete account system
Day 5	Create the Task model and add-task form	Logged-in users can add tasks
Day 6	Display each user's task list from the database	Tasks show on the page
Day 7	Update and delete tasks (mark complete, remove)	Full task management features
Day 8	Restrict tasks so users only see their own; styling	Secure, styled application
Day 9	Testing and bug fixes	Polished, tested application
Day 10	Deploy online and write documentation	Live website + final write-up